



ЛЕКЦИЯ 16

Большие языковые модели в системах безопасности

Возможности, ограничения и риски

Машинное обучение и безопасность систем

Магистратура НИУ ВШЭ (МИЭМ) · ОП «Информационная безопасность и технологии ИИ»

Автор: Силаев Ю. В. Время изучения: ~75 минут · 57 слайдов



О чём эта лекция за одну минуту

2 / 57

ЧАСТЬ 1 · ВВЕДЕНИЕ

Краткая карта: что вы прочитаете и зачем

Большие языковые модели (LLM) уже применяются в работе центров мониторинга безопасности (SOC): они суммируют описания инцидентов, помогают разбирать логи, классифицируют тикеты и ищут ответы по внутренней базе знаний. Эта лекция показывает, **где LLM действительно полезны, а где их применение рискованно** – и почему «убедительный ответ» не равен «корректному ответу».

СЮЖЕТ 1

LLM как инструмент

Помощник аналитика: суммаризация, классификация, извлечение, поиск ответа. Ускоряет работу, но ошибается и не проверяет себя сам.

СЮЖЕТ 2

LLM как агент

Модель вызывает внешние инструменты (поиск, SIEM, тикеты). Удобнее – но получает реальные действия в инфраструктуре и расширяет модель угроз.

***Сквозная рамка лекции:** LLM – это компонент системы, а не самостоятельное решение. Полезность измеряется ускорением аналитика, а ответственность за вывод остаётся на инженере и протоколе проверки.*



Учебные результаты

- объяснять, в каких задачах безопасности LLM полезны, а в каких их применение сомнительно;
- различать режимы использования: генерация, суммаризация, классификация, извлечение, поиск по знаниям, помощь аналитику;
- называть ключевые ограничения LLM: галлюцинации, нестабильность, ложная уверенность, чувствительность к формулировке запроса;
- объяснять, почему оценивать LLM труднее, чем обычный классификатор;
- понимать риски использования внешнего контекста, retrieval и инструментов расширения возможностей модели;
- различать модель как помощника аналитика и модель как автоматического исполнителя решения;
- описывать, как переход от «LLM-чата» к «LLM-агенту» расширяет поверхность атаки (tool use, RAG-источники, инструментальные интерфейсы);
- формулировать базовые требования безопасного и ответственного использования LLM в ИБ-сценариях.



Что нужно знать до начала

4 / 57

ЧАСТЬ 1 · ВВЕДЕНИЕ

Предварительные требования

Лекция самостоятельна и **не предполагает отдельного курса по обработке естественного языка**. Достаточно того, что вы уже изучили в курсе. Ниже – на какие лекции опирается материал и что именно из них понадобится.

L13

Основы нейронных сетей – Общая структура нейросети и логика обучения – LLM это тоже нейросеть.

L15

Тексты и эмбединги – Векторные представления текста (embeddings) и упоминание трансформеров – фундамент LLM и retrieval.

L03

Постановка ML-задач в ИБ – Корректная формулировка задачи; здесь – постановка задачи для LLM-компонента.

Не требуется заранее: внутреннее устройство трансформера, pretraining/fine-tuning, оптимизация инференса – в курсе не разбираются и для понимания лекции не нужны.



План лекции

5 / 57

ЧАСТЬ 1 · ВВЕДЕНИЕ

Семь частей · 57 слайдов · ~75 минут самостоятельного изучения

№	Часть лекции	Слайды	Время
1	Введение	1-5	~6 мин
2	Мотивация: зачем LLM в SOC	6-10	~6 мин
3	Основное содержание (4 блока)	11-48	~46 мин
4	Связки с практикой ИБ	49-52	~5 мин
5	Итоги	53-55	~4 мин
6	Источники для углубления	56	~1 мин
7	Глоссарий	57	~2 мин

Часть 3 – ядро лекции: четыре смысловых блока (LLM на прикладном уровне · ограничения и оценка · RAG · агентские паттерны и модель угроз).



Проблема не в нехватке данных, а в нехватке внимания аналитика

Центр мониторинга безопасности (SOC) тонет в тексте: тысячи алертов в сутки, очереди тикетов, многостраничные описания инцидентов, гигабайты логов и командных строк. Узкое место – **не вычисления, а человеческое внимание**. Аналитик физически не успевает прочитать и сопоставить всё, что генерирует инфраструктура.

Сжать

Суммаризация длинных описаний инцидентов и тредов до сути за секунды.

Разобрать

Помощь в чтении логов, командных строк, незнакомых артефактов.

Найти

Поиск похожих кейсов и ответов по внутренней базе знаний SOC.

***Ключевая идея:** LLM не принимает решение об инциденте – она ускоряет аналитика, готовя черновик, который человек проверяет и утверждает. Польза измеряется сэкономленным временем, а не «правильными ответами без участия человека».*



Реалистичный сценарий, где LLM экономит часы рутины

ИБ-КЕЙС

Очередь triage в ночную смену

Один дежурный аналитик, смена 12 часов, **≈400 новых тикетов** от SIEM. Большинство рутинные (сработки сигнатур, шумные источники), но среди них могут прятаться единичные реальные инциденты.

Без помощника аналитик читает каждый тикет вручную: 1-2 минуты на описание. 400 тикетов – это 7-13 часов чистого чтения, почти вся смена.

LLM-помощник готовит для каждого тикета черновик: резюме в 2-3 строки, предполагаемую категорию и список похожих прошлых кейсов. Аналитик проверяет черновик и углубляется только там, где нужно.

ЭФФЕКТ НА РУТИНЕ

Без LLM

7-13 ч

только чтение очереди

С LLM-черновиком

1.5-3 ч

проверка резюме + углубление

Важно: выигрыш – в скорости рутины, а не в замене аналитика. Решение по каждому потенциальному инциденту остаётся за человеком; LLM лишь сокращает объём ручного чтения.



Главное заблуждение, ради которого написана вся лекция

LLM отвечает связно, уверенно и по делу – поэтому возникает естественное ощущение, что она «понимает» задачу и ей можно доверять как опытному коллеге. Это ощущение и есть ловушка.

ЧАСТАЯ ОШИБКА

«Звучит убедительно – значит, верно»

Студенты и инженеры переносят на LLM интуицию из общения с людьми: у человека уверенный, грамотный, структурированный ответ обычно коррелирует с компетентностью. У языковой модели этой связи нет – **беглость текста и его правильность порождаются независимо.**

Почему это особенно опасно в ИБ: аналитик принимает решения под давлением времени. Убедительный, но неверный черновик резюме инцидента может направить расследование по ложному следу – и чем грамотнее звучит ответ, тем меньше желание его перепроверять.

Этот тезис – стержень лекции: «убедительно» и «корректно» – два независимых свойства ответа. Всё остальное (проверка, ограничение области применения, human-in-the-loop) вытекает отсюда.



Типовые провалы, когда LLM встраивают «как есть», без протокола проверки

Если относиться к LLM как к готовому решению, а не как к компоненту системы, появляются предсказуемые отказы. Ниже – три, которые регулярно встречаются в реальных внедрениях.

ПРОВАЛ 1

Ложный след в расследовании

Модель выдумывает несуществующий CVE или неверную атрибуцию – аналитик идёт не за тем инцидентом.

ПРОВАЛ 2

Утечка чувствительных данных

В запрос к внешней модели вставляют логи с секретами, внутренними адресами, перс. данными.

ПРОВАЛ 3

Тихая деградация качества

Оценили по паре удачных примеров; на потоке доля ошибок выше, но её никто не измеряет.



Рамка лекции: модель как компонент системы

10 / 57

ЧАСТЬ 2 · МОТИВАЦИЯ

Сквозная линия курса, в которую встроена эта лекция

Курс последовательно проводит одну мысль: **ML-модель – это часть системы, а не самостоятельный «умный блок», которому можно делегировать решение.** L16 – очередная точка на этой линии. Для LLM она реализуется через human-in-the-loop: модель готовит, человек проверяет и отвечает за результат.



Что это значит для слайдов дальше: каждую возможность LLM мы будем сразу проверять вопросом «кто отвечает за результат – модель или человек?». Ответ почти всегда: человек, а модель ускоряет его работу.



БЛОК 1 ИЗ 4

LLM на прикладном уровне

Что такое большая языковая модель на уровне, достаточном для инженера ИБ: без устройства трансформера, но с ясным пониманием prompt, few-shot и того, что модель на самом деле делает.

В этом блоке: что такое LLM · prompt как постановка задачи · in-context learning и few-shot · пять режимов использования · пример классификации тикета · граница «генерация против классификации».



Что языковая модель делает на самом деле – и чего не делает

Большая языковая модель – это **та же нейросеть, что и в L13**, только очень большая и обученная на огромных объёмах текста. Её базовая операция предельно проста: по уже имеющемуся тексту она предсказывает, какой **следующий фрагмент (токен)** наиболее вероятен, и так – слово за словом – порождает ответ. Текст она представляет векторами – **эмбедингами из L15**.

ИЗ ЭТОГО СЛЕДУЕТ

- Модель отлично улавливает форму и стиль текста: связность, грамматику, типичные формулировки.
- Она обобщает закономерности языка, а не хранит «базу проверенных фактов».
- Ответ – это правдоподобное продолжение, а не результат проверки истинности.

ЧЕГО МОДЕЛЬ НЕ ДЕЛАЕТ

- Не «понимает» задачу так, как человек, и не осознаёт, что может ошибаться.
- Не сверяется с источником истины, если он не дан явно во входе.
- Не гарантирует один и тот же ответ на один и тот же вопрос.

Устройство трансформера, обучение и инференс в курсе не разбираются – для работы с LLM как с компонентом системы достаточно этой прикладной модели.



Рабочее определение для целей курса

ОПРЕДЕЛЕНИЕ

Большая языковая модель (LLM)

Большая нейросетевая модель, обученная на больших корпусах текста предсказывать следующий токен и за счёт этого порождающая связный текст в ответ на запрос. Применяется как **универсальный текстовый преобразователь**: вход – текст (запрос и контекст), выход – текст (ответ).

Ключевые свойства этого определения

Текст → текст

Любая задача сводится к формулировке «дай на вход текст, получи на выходе текст».

Общая, не специализированная

Одна модель решает много разных задач без отдельного обучения под каждую.

Вероятностная по природе

Выход – наиболее правдоподобное продолжение, а не доказанно верный ответ.



Как именно мы говорим модели, что от неё нужно

У обычного классификатора задача «зашита» в обученную модель. У LLM задача задаётся **текстом запроса (prompt)** прямо во время использования: промпт – это сразу и инструкция, и контекст, и формат ответа. По сути это **постановка задачи (как в L03)** на естественном языке, а не в коде и обучении.

Из чего складывается хороший prompt

Инструкция. что сделать: «классифицируй тикет», «суммируй инцидент в 3 строки».

Контекст. данные для работы: текст тикета, фрагмент лога, выдержка из базы знаний.

Формат ответа. в каком виде вернуть результат: метка класса, JSON, маркированный список.

Ограничения. чего не делать: «не придумывай CVE», «если данных нет – так и скажи».

Меняя только формулировку промпта, мы меняем поведение системы без переобучения – это и сила, и источник нестабильности (слайд 23).



Как «обучать» модель задаче примерами – без обучения весов

ОПРЕДЕЛЕНИЕ

In-context learning и few-shot

In-context learning – способность LLM подстроиться под задачу прямо из примеров в самом запросе, без изменения весов модели. **Few-shot prompting** – частный приём: в промпт добавляют несколько примеров вида «вход → правильный ответ», и модель продолжает в том же духе для нового входа. **Zero-shot** – без примеров, только инструкция.

ZERO-SHOT

Инструкция: классифицируй тикет.

Тикет: «...»

→ Ответ: ?

FEW-SHOT (2 ПРИМЕРА)

Тикет: «...» → Phishing

Тикет: «...» → Malware

Тикет: «...» → ?

Примеры в промпте задают и формат, и желаемое поведение. Но это не настоящее обучение: модель ничему не «научается» за пределами одного запроса.



Одна модель, разные задачи – и разная цена ошибки в каждой

Как универсальный текстовый преобразователь LLM закрывает несколько разных классов задач. В ИБ важны пять. Они различаются не только тем, что делают, но и тем, насколько критична ошибка и насколько легко её заметить.

Режим	Что делает	Пример в ИБ
Summarization	сжать длинный текст до сути	резюме многостраничного описания инцидента
Classification	отнести текст к категории	категория SOC-тикета: phishing / malware / шум
Extraction	вытащить сущности из текста	IP, домены, хеши и CVE из текста алерта
Question answering	ответить на вопрос по тексту	поиск ответа по внутренней базе знаний SOC
Reasoning assist.	помочь рассуждать по шагам	разбор подозрительной командной строки

***Держите в голове:** extraction и QA легко проверить (есть ли IP в тексте, есть ли ответ в источнике), а reasoning и summarization – труднее: правдоподобный текст не значит верный. К оценке вернёмся в Блоке 2.*



Few-shot prompting на конкретном SOC-тикете, шаг за шагом

PROMPT (FEW-SHOT, 2 ПРИМЕРА)

Классифицируй SOC-тикет в один из классов: Phishing, Malware, Noise. Если неясно – верни Uncertain.

Тикет: «Массовая рассылка писем со ссылкой на поддельный портал входа»

→ **Phishing**

Тикет: «EDR: запуск powershell с закодированной командой -enc»

→ **Malware**

Тикет: «Пользователь сообщил о письме с просьбой срочно сменить пароль по ссылке во вложении»

→ ?

1 · Инструкция

задаёт классы и поведение по умолчанию (Uncertain при неясности).

2 · Примеры

два размеченных тикета задают формат «текст → класс».

3 · Новый вход

тикет без метки; модель продолжает паттерн.

4 · Ожидаемо

Phishing – письмо со ссылкой на смену пароля.

Один пример – это демонстрация, не доказательство качества. Почему так – слайд 26.



Самая частая путаница при первом применении LLM

ЧАСТАЯ ОШИБКА

«Вернула метку – значит, классификатор»

LLM может вернуть слово «Phishing», и кажется, что перед нами готовый классификатор. Но модель **генерирует правдоподобный текст**, а не вычисляет откалиброванную вероятность класса. У неё нет порога решения, нет калибровки, а «уверенность» в тоне ответа не связана с реальной частотой ошибок.

ПОЧЕМУ ЭТО НЕВЕРНО

Чего нет у сгенерированной метки

Нет вероятности и порога. Нельзя сдвинуть решение под цену ошибки (как в L04/L07) – модель не выдаёт калиброванный P(класс).

Нет стабильности. Перефразируйте тикет – и метка может измениться, хотя суть та же.

Нет гарантии множества классов. Модель может вернуть класс не из списка или выдумать новый, если её прямо не ограничить.

Правильная рамка: нужна надёжная классификация с контролем ошибок – это задача обычного классификатора (L06-L08). LLM хороша там, где выход – текст для человека.



Что нужно унести из «LLM на прикладном уровне»

1

LLM – это нейросеть, предсказывающая токены

Та же модель, что в L13, на эмбедингах из L15; вход – текст, выход – текст.

2

Prompt – это постановка задачи на языке

Инструкция + контекст + формат + ограничения; задаётся при использовании, без переобучения.

3

Few-shot задаёт поведение примерами

Примеры «вход → ответ» в запросе; но это не настоящее обучение модели.

4

Пять режимов различаются ценой ошибки

Summarization, classification, extraction, QA, reasoning – проверяемость у них разная.

5

Генерация – не надёжная классификация

Сгенерированная метка не имеет вероятности, порога и стабильности.

Дальше – Блок 2: почему LLM ошибается незаметно и почему её так трудно оценивать.



БЛОК 2 ИЗ 4

Ограничения LLM

Почему беглый, уверенный ответ LLM нельзя принимать на веру: галлюцинации, нестабильность при перефразировке, ложная уверенность — и почему из-за всего этого LLM так трудно оценивать.

В этом блоке: что такое галлюцинация · галлюцинации в ИБ · нестабильность ответа · пример с перефразировкой · ложная уверенность · почему оценка LLM трудна · подходы к оценке и red teaming.



Главное ограничение, с которым придётся жить всегда

ОПРЕДЕЛЕНИЕ

Галлюцинация модели

Уверенно сформулированный ответ, который **звучит правдоподобно, но не соответствует действительности или не выводится из данных**, переданных модели. Модель не «лжёт» намеренно – она порождает наиболее вероятное продолжение текста, и иногда самое вероятное оказывается вымыслом.

Почему это происходит – и почему это не баг, а свойство

Модель оптимизирует правдоподобие

Цель при обучении – связный, вероятный текст, а не проверенная истина.

Нет встроенной проверки фактов

Если источник истины не дан во входе, сверяться модели не с чем.

Пробелы заполняются догадкой

Там, где данных мало, модель «достраивает» правдоподобную деталь.



Где выдуманная деталь стоит особенно дорого

В безопасности цена галлюцинации выше, чем в большинстве задач: аналитик действует по выводу модели, а ошибочная деталь выглядит так же убедительно, как верная. Три типичных случая:

Выдуманный CVE или сигнатура

Модель ссылается на несуществующий CVE-идентификатор или «известную» уязвимость, которой нет. Аналитик ищет патч, которого не существует.

Несуществующая команда или флаг

При разборе командной строки модель уверенно объясняет флаг, которого у утилиты нет, или придумывает «безопасное» назначение вредоносной команды.

Ложная атрибуция

Модель приписывает активность конкретной АPT-группе или инструменту на основании поверхностного сходства – расследование уходит в неверную сторону.

Общее: галлюцинацию не отличить от правды по форме ответа – её ловят проверкой против источника (далее).



Тот же вопрос – другой ответ: два источника недетерминизма

У обычного классификатора одинаковый вход даёт одинаковый выход. У LLM это не гарантировано: ответ может меняться от запуска к запуску и – особенно – от **формулировки запроса**. Два независимых источника нестабильности:

1 · СЛУЧАЙНОСТЬ ГЕНЕРАЦИИ

Модель выбирает следующий токен вероятностно. При ненулевой «температуре» один и тот же запрос даёт разные ответы. Это можно уменьшить, но не всегда убрать полностью.

2 · ЧУВСТВИТЕЛЬНОСТЬ К PROMPT

Малое изменение формулировки – синоним, порядок предложений, лишняя деталь – может изменить вывод, хотя смысл задачи тот же. Это коварнее: выглядит как «случайная» ошибка, а на деле системная.

Следствие для ИБ: нельзя проверить систему на одной формулировке и считать, что она работает. Нужен набор вариантов запроса – см. оценку дальше.



Пример: перефразировка

24 / 57

ЧАСТЬ 3 · ОСНОВНОЕ · БЛОК 2

Один тикет, три формулировки запроса – три разных вердикта

Один и тот же SOC-тикет про подозрительный вход в систему. Меняем только обёртку запроса – суть остаётся. Гипотетический, но типичный результат:

Формулировка А

«Классифицируй тикет: подозрительный или нет»



Подозрительный

Формулировка В

«Это рутинное событие? Ответь да/нет»



Да, рутинное

Формулировка С

«Оцени риск тикета по шкале 1-5»



Риск 2 из 5

ЧТО НЕ ТАК

Формулировка В дала противоположный вывод. На автоматическом режиме реальный инцидент был бы закрыт как шум – из-за одной лишь обёртки запроса.



Почему уверенный тон – не сигнал достоверности

LLM почти всегда отвечает уверенно: грамотно, структурно, без оговорок – даже когда ошибается. **Тон ответа и его правильность порождаются независимо** (мы видели это в мотивации). У человека неуверенность обычно «слышна»; у модели её нет, поэтому самоконтроль читателя отключается.

У ЧЕЛОВЕКА-ЭКСПЕРТА

Сомнение видно: оговорки, «кажется», «надо проверить». Уверенность грубо коррелирует с компетентностью.

У ЯЗЫКОВОЙ МОДЕЛИ

Тон одинаково уверенный и для верного, и для выдуманного ответа. «Уверенность» нельзя читать как вероятность правоты.

***Практический вывод:** если просить модель «оценить свою уверенность», эта самооценка тоже сгенерирована и тоже может быть неверной. Доверие должно опираться на внешнюю проверку, а не на тон.*



Почему «у меня сработало» ничего не говорит о качестве

ЧАСТАЯ ОШИБКА

«Я проверил на паре тикетов – работает»

Инженер прогоняет два-три удачных примера, видит верный ответ и делает вывод о качестве системы. Но удачная демонстрация **не отличается по форме от случайного попадания**: на следующих десятках входов доля ошибок может быть совсем другой, а из-за нестабильности (слайд 23) даже тот же вход даст другой результат.

ПОЧЕМУ ЭТО НЕВЕРНО

Чем демонстрация отличается от оценки

Нет выборки. Несколько примеров не репрезентативны; реальный поток тикетов разнообразнее.

Нет негативных случаев. Удачные примеры выбирают неосознанно – трудные и пограничные не попадают в проверку.

Нет числа. «Работает» – это не метрика. Нужна доля верных на фиксированном наборе, а не впечатление.

Та же дисциплина, что и в обычном ML: вывод о качестве – только по заранее заданному набору, а не по удачным запускам.



Почему оценка LLM трудна

27 / 57

ЧАСТЬ 3 · ОСНОВНОЕ · БЛОК 2

Сравнение с привычной оценкой классификатора из L04

У обычного классификатора оценка отлажена: фиксированный тест, метки, метрики из L04. С LLM так не получается – мешают сразу четыре свойства.

	Классификатор (L04)	LLM
Выход	класс или вероятность	свободный текст – много «правильных» форм
Эталон	однозначная метка	часто нет единственно верного ответа
Метрика	accuracy, F1, ROC-AUC	нет одной общепринятой метрики качества
Стабильность	детерминированная	ответ плавает от запуска и формулировки

Вывод: оценка LLM – это не одна цифра, а процедура: набор тест-кейсов, критерии качества и человек, который судит спорные случаи. Как её построить – на следующем слайде.



Четыре опоры вместо одной метрики

Раз одной цифры нет, надёжность собирают из нескольких процедур. Минимальный набор для ИБ-сценария:

- 1 Набор тест-кейсов.** Фиксированный список входов с ожидаемым поведением – включая трудные, пограничные и заведомо «ловушечные». Прогоняется при каждом изменении промпта или модели.
- 2 Рубрики качества.** Явные критерии «хорошего» ответа (точность фактов, полнота, формат, отсутствие выдумок). Превращают расплывчатое «нравится» в проверяемый чек-лист.
- 3 Экспертная проверка.** Человек-аналитик оценивает спорные и важные случаи. Незаменима там, где нет однозначного эталона, а цена ошибки высока.
- 4 Red teaming.** Целенаправленный поиск входов, на которых система ломается: перефразировки, провокации, попытки сбить с инструкции. Определение – на следующем слайде.



Определение: red teaming

29 / 57

ЧАСТЬ 3 · ОСНОВНОЕ · БЛОК 2

Систематический поиск провалов – в безопасной учебной рамке

ОПРЕДЕЛЕНИЕ

Red teaming LLM

Целенаправленная проверка системы на устойчивость: команда заранее ищет входы, на которых модель **даёт неверный, небезопасный или нежелательный ответ** – перефразировки, провокации, противоречивые инструкции, попытки заставить нарушить заданные ограничения. Цель – найти слабые места до того, как их найдут в эксплуатации.

ЧТО ДЕЛАЕМ В КУРСЕ

Проверяем свою же систему: подбираем трудные тикеты, перефразируем запросы, смотрим, где вывод плывёт. Защитная, учебная рамка.

ЧЕГО НЕ ДЕЛАЕМ

Не разрабатываем рабочие атаки и jailbreak-техники. Offensive-эксплуатация вне курса; цель – оборона, а не нападение.

Полноценный adversarial-сюжет (prompt injection как атака) – в L18.



Что нужно унести из «Ограничения LLM»

1

Галлюцинация – правдоподобный вымысел

Уверенный ответ, не выводимый из данных; это свойство модели, а не редкий сбой.

2

Ответ нестабилен

Меняется от запуска и от формулировки запроса; одной проверки на одном промпте мало.

3

Уверенный тон ≠ достоверность

Тон и правильность независимы; самооценка модели тоже может быть выдумана.

4

Один пример – не доказательство

Вывод о качестве – только по фиксированному набору входов, а не по удачным запускам.

5

Оценка LLM – процедура, не метрика

Тест-кейсы + рубрики + эксперт + red teaming; одной цифры качества не существует.

Дальше – Блок 3: как дать модели внешние знания через retrieval (RAG) – и какие риски это приносит.



БЛОК 3 ИЗ 4

Retrieval-Augmented Generation

Как дать модели нужные знания прямо в запросе вместо того, чтобы полагаться на её память — и почему это решает одни проблемы, но создаёт новые.

В этом блоке: зачем RAG · определение RAG · схема пайплайна · ИБ-кейс с базой знаний SOC · зависимость от качества источников · contamination и prompt injection через контекст.



Три проблемы памяти модели, которые retrieval решает

Сама по себе LLM отвечает из того, что «впитала» при обучении. Для ИБ-задач этого мало по трём причинам – и все три снимаются, если **нужные данные передать модели прямо в запросе**, а не надеяться, что она их «помнит».

Устаревание

Знания модели зафиксированы на момент обучения. Свежий CVE, вчерашний инцидент, новый playbook – она не знает.

Нет приватных знаний

Внутренняя база знаний SOC, ваши тикеты и регламенты не входили в обучение – модель про них не в курсе.

Ограничение контекста

За один запрос модель видит ограниченный объём текста – всю базу не вместить, нужно подать только релевантное.

Идея RAG в одной фразе: не спрашивай модель «что ты помнишь?», а дай ей источник и спроси «что здесь сказано?».



Архитектурная идея, без деталей реализации

ОПРЕДЕЛЕНИЕ

Retrieval-Augmented Generation

Подход, при котором перед генерацией ответа система **находит релевантные фрагменты во внешнем источнике и добавляет их в запрос** как контекст. Модель отвечает, опираясь на найденный текст, а не только на свою память. Поиск обычно идёт по эмбедингам (как в L15): запрос и документы переводятся в векторы, и ищутся ближайшие.

ЧТО RAG ДАЁТ

Свежие и приватные знания, ответ со ссылкой на источник, возможность проверить ответ против поданного текста.

ЧЕГО RAG НЕ ДАЁТ

Гарантии правды: если источник плохой или поиск нашёл не то – модель уверенно ответит на основе мусора.

RAG переносит вопрос доверия с «памяти модели» на «качество источника и поиска» – это и разберём дальше.



Концептуальный пайплайн: запрос → поиск → контекст → ответ

Четыре шага. Реализацию (векторные базы, индексы, чанкинг) в курсе не разбираем – важна логика потока данных и места, где он может сломаться.



Ключевая точка контроля – шаг 3: ответ хорош ровно настолько, насколько релевантен и достоверен поданный контекст. Мусор на входе шага 3 = уверенный мусор на выходе.



RAG-помощник по внутренним playbook'ам и прошлым инцидентам

ИБ-КЕЙС

Вопрос-ответ по внутренней базе знаний

В SOC накоплены сотни playbook'ов, регламентов и разборов прошлых инцидентов. Новый аналитик не знает, где что лежит.

RAG-помощник: на вопрос «как реагировать на срабатывание X?» система находит релевантные playbook'и и прошлые похожие кейсы и формирует ответ со ссылками на конкретные документы.

Польза: ответ опирается на реальные внутренние документы (не на память модели) и его можно проверить, открыв источник.

ГДЕ ЭТО ЛОМАЕТСЯ

Поиск нашёл не то

релевантного документа нет, но поиск вернул похожий – ответ уверенный и неверный.

Документ устарел

playbook не обновляли; модель добросовестно цитирует устаревшую процедуру.

Ответа нет в базе

если модель не ограничить, она «достроит» ответ из памяти – это уже галлюцинация.

Вывод кейса: RAG не устраняет проверку – он переносит её на источник. Ответ всё равно подтверждает аналитик.



RAG не делает ответ верным – он делает его настолько верным, насколько верен источник

RAG смещает риск, но не убирает его. Раньше модель могла выдумать факт; теперь она **уверенно перескажет то, что нашла** – даже если найденное неполно, устарело или просто не относится к вопросу. Три типа проблем источника:

Неполнота

В базе есть только часть ответа. Модель отвечает по фрагменту, не предупреждая, что картина неполная.

Устаревание

Источник верен на момент написания, но процедура или данные изменились. Ответ формально «из документа», но уже неверный.

Нерелевантность

Поиск вернул похожий по словам, но не по смыслу фрагмент. Модель честно отвечает не на тот вопрос.

Поэтому к источнику для RAG относятся как к части системы: следят за актуальностью, происхождением и тем, что именно попадает в индекс.



Когда источник не просто плохой, а враждебный

До сих пор источник был честным, но несовершенным. Но контекст приходит из внешнего мира – а значит, в него может попасть **текст, специально составленный, чтобы повлиять на модель**. Две концепции на уровне идеи:

ПОНЯТИЕ 1

Contamination контекста

В источник попадают недостоверные, мусорные или подделанные данные. Поиск их извлекает, и модель отвечает на основе заражённого фрагмента, не отличая его от настоящего.

ПОНЯТИЕ 2

Prompt injection через контекст

В найденном тексте спрятана инструкция модели («игнорируй прежние указания и...»). Модель может принять её за команду – поведение системы меняет посторонний документ.

Здесь только ставим проблему. Полноценный adversarial-сюжет – prompt injection как класс атак, модель угроз и защита – разбирается в L18. Для RAG достаточно запомнить: контекст из внешнего мира не доверенный по умолчанию.



Что нужно унести из «Retrieval-Augmented Generation»

1

RAG даёт модели знания в запросе

Находим релевантный фрагмент во внешнем источнике и кладём в prompt – свежие и приватные данные без переобучения.

2

Поиск идёт по эмбедингам

Запрос и документы – в векторы (L15), ищем ближайшие. Реализацию в курсе не разбираем.

3

RAG переносит риск, а не убирает

Доверие смещается с памяти модели на качество источника и поиска.

4

Качество источника решает всё

Неполный, устаревший или нерелевантный фрагмент → уверенный неверный ответ.

5

Контекст не доверенный по умолчанию

Contamination и prompt injection через источник; как атака – в L18.

Дальше – Блок 4: что меняется, когда LLM не просто отвечает, а вызывает инструменты – переход к агентам.



БЛОК 4 ИЗ 4

От чата к агенту

Что меняется, когда LLM не просто отвечает текстом, а вызывает внешние инструменты и выполняет действия. Главный тезис блока: это не «то же самое, но удобнее» – это качественно другая модель угроз.

***В этом блоке:** что такое агент и tool use · MCP-подобные интерфейсы · ИБ-кейс агента-помощника · расширение поверхности атаки · четыре новых вектора · human-in-the-loop vs автоматическое исполнение.*



Когда модель перестаёт только говорить и начинает действовать

ОПРЕДЕЛЕНИЕ

LLM-агент и tool use

LLM-агент – это модель, которой дали возможность **вызывать внешние инструменты** (tool use): поиск, обращение к базе, чтение тикетов, запрос к SIEM. Модель не только генерирует текст, но и решает, какой инструмент вызвать, с какими параметрами, и использует результат для следующего шага – выстраивая **цепочку вызовов** до достижения цели.

LLM КАК ЧАТ

Вход – текст, выход – текст. Модель ничего не делает во внешнем мире; всё, что она может, – это сказать. Действует человек.

LLM КАК АГЕНТ

Выход может быть действием: вызвать инструмент, изменить данные, отправить запрос. Модель получает рычаги в реальной системе.

Этот переход – от «сказать» к «сделать» – и есть источник новых рисков. Разработку агентов в курсе не углубляем.



Как модели стандартно «дают» инструменты – идея инструментальных протоколов

Чтобы агент мог вызывать инструменты, нужен общий способ их описывать и подключать. В 2026 году это оформилось в **инструментальные интерфейсы – протоколы вроде МСП (Model Context Protocol)**. Идея: инструмент описывается единообразно (что делает, какие параметры), и любой агент может его подключить, не зная деталей реализации.

Что это меняет на практике

Подключаемость. Новый инструмент добавляется как «плагин» – агент быстро получает доступ к новым действиям.

Единый язык вызова. Модель работает с инструментами через общий протокол, а не через разрозненные интеграции.

Рост возможностей = рост поверхности. Чем больше подключено инструментов, тем больше действий доступно – и тем шире поверхность атаки.

Важно не уйти в обзор инструментов: для курса достаточно идеи «есть стандартный способ дать модели рычаги». Конкретные продукты не разбираем.



Агент, действующий в инфраструктуре SOC от лица аналитика

ИБ-КЕЙС

Агент-помощник в triage инцидента

Аналитик получает алерт и поручает агенту собрать контекст. Агент сам решает, что нужно, и вызывает инструменты:

- читает тикет в трекере;
- запрашивает связанные события в SIEM;
- ищет похожие инциденты в базе знаний;
- формирует черновик сводки с рекомендацией.

Удобно: то, что аналитик делал руками в пяти системах, агент собирает за один запрос.

ЧТО ИЗМЕНИЛОСЬ vs ЧАТ

Реальный доступ

у агента есть права читать тикеты и обращаться к SIEM – это действия, не текст.

Автономные решения

агент сам выбирает, какой инструмент вызвать и с какими параметрами.

Цепочка шагов

результат одного вызова влияет на следующий – ошибка распространяется.

Тот же удобный сценарий – но теперь у модели есть рычаги в инфраструктуре. Что это значит для угроз – дальше.



Почему переход к агенту качественно меняет модель угроз

У чата худший исход – **неверный текст**, который человек ещё прочитает и оценит. У агента худший исход – **неверное действие в реальной системе**, которое уже выполнено. Это не количественная, а качественная разница: добавляется целый новый класс рисков.

ПОВЕРХНОСТЬ АТАКИ: ЧАТ

- Вход модели – запрос пользователя.
- Выход – текст для человека.
- Худший исход – человека ввели в заблуждение.

ПОВЕРХНОСТЬ АТАКИ: АГЕНТ

- Вход – запрос + данные от инструментов и источников.
- Выход – действия в системах (чтение, запросы, изменения).
- Худший исход – выполнено вредное действие.

Каждый подключённый инструмент и каждый источник контекста – новая точка входа для атакующего. Конкретные векторы – на следующем слайде.



Что добавляется к модели угроз именно при агентских паттернах

Когда у модели появляются инструменты и внешний контекст, открываются классы атак, которых у простого чата нет. Четыре характерных:

1 · Prompt injection через инструмент

Инструмент возвращает данные, в которые вложена инструкция модели. Агент исполняет её как команду – атака приходит не от пользователя, а из ответа инструмента.

2 · Exfiltration через retrieval

Через retrieval или вызов инструмента агента заставляют вытащить и передать наружу данные, к которым у него есть доступ, но не должно быть у запросившего.

3 · Цепочечная атака

Инструмент с побочным эффектом (запись, отправка, удаление) вызывается по подсказке из заражённого контекста – одно действие запускает цепочку последствий.

4 · Неконтролируемые действия

Агент сам выбирает шаги; ошибка или манипуляция приводит к действию, которого никто явно не санкционировал, – и оно уже выполнено.

Объединяет их одно: вход и действия агента выходят за пределы доверенного диалога с пользователем.



Самое опасное заблуждение всего блока

ЧАСТАЯ ОШИБКА

«Агент – то же самое, просто удобнее»

Кажется, что агент – это чат с надстройкой: те же ответы, только сам ходит за данными. Поэтому к нему применяют те же меры доверия, что и к чату. Но добавление инструментов **качественно расширяет поверхность атаки** – появляется целый класс рисков (слайд 44), которого у чата не было.

ПОЧЕМУ ЭТО НЕВЕРНО

Чат и агент – разные модели угроз

Другой вход. У агента во вход попадают данные инструментов и источников – не только запрос пользователя.

Другой выход. Выход агента – действия в системах, а не текст, который человек ещё прочитает.

Другая цена ошибки. Неверное действие уже выполнено; «перечитать и не поверить» поздно.

Это и есть threat modeling для ML-компонента – сквозная линия курса. Систематически модель угроз для ML (включая агентов) разбирается в L18.



Ключевое различие: помощник или исполнитель решения

ОПРЕДЕЛЕНИЕ

Human-in-the-loop vs fully automated

Human-in-the-loop (HITL) – режим, в котором **человек утверждает значимое действие или вывод модели** перед его применением. Fully automated – модель (или агент) выполняет действие самостоятельно, без проверки человеком. Различие не в технологии, а в том, **кто несёт ответственность за результат**.

HUMAN-IN-THE-LOOP

Модель = помощник.

Готовит черновик, предлагает, собирает контекст.

Решение и ответственность – на человеке. Ошибка ловится до применения.

FULLY AUTOMATED

Модель = исполнитель.

Действие применяется без проверки. Ошибка или манипуляция реализуется сразу. Допустимо только для обратимых, малорисковых действий.



Возврат к сквозной линии: модель как компонент системы

Лекция началась с тезиса «LLM – компонент системы, а не самостоятельное решение». Для агентов он становится жёстким требованием: **чем выше риск и необратимость действия, тем обязательнее человек в контуре**. Автоисполнение рекомендаций LLM без проверки особенно опасно именно в безопасности.

Грубое правило: что можно автоматизировать, а что – нет

Можно ближе к авто

Обратимые, малорисковые, легко проверяемые действия: пометить тикет, собрать контекст, предложить черновик.

Только через человека

Необратимые или дорогие действия: блокировка, изоляция хоста, сброс доступа, рассылка во вне, изменение конфигурации.

Линия «модель как компонент» (L03 → L07 → L16 → L18 → L20) здесь достигает агентов: ответственность остаётся на человеке и системе вокруг модели.



Что нужно унести из «От чата к агенту»

1

Агент = LLM + вызов инструментов

Модель не только отвечает, но и действует: tool use, цепочки вызовов, MCP-подобные интерфейсы.

2

Переход к агенту меняет модель угроз

Это не «удобнее», а качественно другой класс рисков – поверхность атаки расширяется.

3

Четыре новых вектора

Prompt injection через инструмент, exfiltration через retrieval, цепочечная атака, неконтролируемые действия.

4

HITL vs fully automated – вопрос ответственности

Помощник готовит, человек утверждает; автоисполнение – только для обратимых малорисковых действий.

5

Чем выше риск – тем обязательнее человек

Необратимые действия (блокировка, изоляция, сброс доступа) – только через проверку человеком.

На этом основное содержание (Блоки 1-4) завершено. Дальше – связи с практикой ИБ: данные, политика и требования.



Что нельзя бездумно передавать внешней модели – и зачем журналировать

Когда запрос уходит во внешнюю модель, всё, что в нём есть, **покидает периметр организации**. Для ИБ это особенно чувствительно: в логах и тикетах легко оказываются секреты, персональные данные и детали инфраструктуры. Два базовых требования:

1 · ОГРАНИЧЕНИЕ ВХОДА

Решить заранее, что в принципе нельзя отправлять: учётные данные и токены, персональные данные, ключи и пароли, точные адреса и топологию сети. Маскировать или удалять до отправки.

2 · ЖУРНАЛИРОВАНИЕ

Фиксировать, кто, когда и с какими данными обращался к модели. Журнал нужен и для расследования инцидентов с самой LLM, и для контроля того, что политика входа соблюдается.

Тип развёртывания (внешний сервис или модель в своём контуре) меняет детали, но не отменяет вопрос «что именно мы отправляем?».



Разбор реального тикета: что маскируем, что оставляем, что логируем

ИБ-КЕЙС

Тикет на суммаризацию во внешней LLM

Исходный тикет содержит вперемешку и полезный контекст, и чувствительные поля:

- описание подозрительной активности – нужно;
- внутренние IP и имя хоста – маскировать;
- логин и сессионный токен – удалить;
- ФИО затронутого пользователя – псевдоним;
- текст и время события – нужно.

После маскирования смысл для суммаризации сохраняется, а утечка – нет.

ПРАВИЛО РЕШЕНИЯ

Нужно для задачи?

если поле не помогает суммаризации – его незачем отправлять.

Чувствительно?

секреты и идентификаторы – удалить или заменить псевдонимом.

Залогировать факт

записать, что и в каком виде ушло в модель.

Маскирование – не формальность: один незамаскированный токен в запросе – это уже утечка секрета за периметр.



Базовые требования к безопасному и ответственному использованию

Свёртка всей лекции в практический список. Перед тем как ставить LLM в ИБ-процесс, пройдите по шести пунктам:



Область применения

Чётко определена задача и где модель НЕ применяется.
Генерация текста, а не надёжное автоматическое решение.



Протокол проверки

Есть набор тест-кейсов, рубрики и человек для спорных случаев. Качество измеряется, а не предполагается.



Контроль источников

Для RAG: актуальность, происхождение и доверенность контекста под контролем.



Human-in-the-loop

Значимые и необратимые действия утверждает человек.
Автоисполнение – только для малорисковых.



Данные и приватность

Чувствительные данные маскируются до отправки; решено, что нельзя передавать в принципе.



Журналирование

Зафиксировано, кто, когда и с какими данными обращался к модели.



Частая ошибка, которая дорого обходится в ИБ

ЧАСТАЯ ОШИБКА

«Отправлю как есть – модель же не запомнит»

Чтобы не возиться с маскированием, в модель отправляют сырой лог или тикет целиком. Рассуждение «это разовый запрос, ничего не случится» **игнорирует сразу несколько рисков**: данные уже покинули периметр, и что с ними дальше – вне вашего контроля.

ПОЧЕМУ ЭТО НЕВЕРНО

Что происходит с отправленными данными

Передача за периметр. Секрет, ушедший во внешний сервис, уже считается раскрытым – отозвать его нельзя.

Логи и обучение. Запрос может сохраняться на стороне сервиса; политика хранения часто вне вашего контроля.

Memorization. Модели способны воспроизводить запомненные фрагменты обучающих данных – privacy-риск (подробно в L20).

Правило простое: маскируй до отправки, а не надейся, что «обойдётся». Стоимость маскирования несопоставима со стоимостью утечки.



Семь ключевых утверждений

- 1 LLM – полезный инструмент в ИБ, но не замена протоколу проверки и инженерному контролю.
- 2 «Убедительно» и «корректно» – независимые свойства ответа; уверенный тон не значит достоверность.
- 3 Галлюцинации, нестабильность и чувствительность к prompt – это свойства модели, а не редкие сбои.
- 4 Оценка LLM – это процедура (тест-кейсы, рубрики, эксперт, red teaming), а не одна метрика.
- 5 RAG переносит доверие с памяти модели на качество источника – и не отменяет проверку.
- 6 Переход к агенту качественно расширяет модель угроз, а не просто «делает удобнее».
- 7 Чем выше риск действия – тем обязательнее human-in-the-loop; ответственность остаётся на человеке.



Быстрый ориентир: зелёные и красные сценарии LLM в ИБ

ХОРОШО ПОДХОДИТ

- Суммаризация описаний инцидентов и длинных тредов.
- Черновик отчёта или сводки для аналитика.
- Извлечение сущностей (IP, домены, хеши) с последующей проверкой.
- Поиск ответа по базе знаний через RAG, со ссылкой на источник.
- Помощь в разборе незнакомых логов и команд – как подсказка, не вердикт.

ПЛОХО ПОДХОДИТ / РИСКОВАННО

- Надёжная классификация с контролем ошибок – это задача обычного классификатора.
- Автоматическое исполнение решений без проверки человеком.
- Задачи, где нужен проверяемый порог и калиброванная вероятность.
- Обработка сырых чувствительных данных без маскирования.
- Любой вывод, который некому проверить, но по которому будут действовать.



Куда эта лекция встроена и что читать дальше

L16 – часть Блока 3 «Надёжность, интерпретируемость, безопасность» и точка на сквозной линии «модель как компонент системы».

ОПИРАЕТСЯ НА (ПОЗАДИ)

L13 – основы нейросетей. **L15** – тексты и эмбединги.

L03 – постановка ML-задач. **L04/L07** – метрики и порог.

ВЕДЁТ К (ВПЕРЕДИ)

L17 – устойчивость и дрейф.

L18 – prompt injection как атака, threat modeling.

L19 – poisoning обучающих данных.

L20 – privacy и memorization.

Сквозная линия «модель как компонент системы»:



Рекомендуемый следующий шаг: L18 – там prompt injection и расширение поверхности атаки из этой лекции разбираются как полноценная модель угроз.



Тема LLM/safety вне учебников ПУД – поэтому специализированные источники

ОБЯЗАТЕЛЬНО

- Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks – первоисточник RAG.
- OWASP Top 10 for LLM Applications (2025) – prompt injection, утечки, риски; разделы LLM01, LLM06.
- NIST AI Risk Management Framework (AI RMF 1.0), разделы Govern и Measure – рамка оценки рисков.

ДОПОЛНИТЕЛЬНО

- Ji et al. (2023). Survey of Hallucination in Natural Language Generation – обзор причин галлюцинаций.
- Anthropic / OpenAI model cards и safety-документация – практика оценки и red teaming.

ТУТОРИАЛЫ / ПРАКТИКА

- Документация по инструментальным интерфейсам (Model Context Protocol) – обзорные материалы.
- Практические гайды по построению наборов тест-кейсов и рубрик для оценки LLM.



Ключевые термины

LLM – большая языковая модель: предсказывает токены, порождает текст по запросу.

Prompt – текст запроса; постановка задачи модели на естественном языке.

Few-shot – примеры «вход → ответ» в запросе, задающие поведение без обучения весов.

Галлюцинация – правдоподобный, но неверный или невыводимый из данных ответ.

RAG – генерация с подстановкой найденного во внешнем источнике контекста.

Retrieval – поиск релевантных фрагментов (обычно по эмбедингам) для контекста.

Contamination – подделанные данные, попавшие в источник контекста.

Prompt injection – скрытая в тексте инструкция, которую модель принимает за команду.

LLM-агент – модель, вызывающая внешние инструменты и выполняющая действия.

Tool use – вызов агентом внешних инструментов (поиск, SIEM, тикеты).

MCP-подобный интерфейс – стандартный протокол подключения инструментов к модели.

Human-in-the-loop – режим, где значимое действие/вывод утверждает человек.

Red teaming – целенаправленный поиск входов, на которых система ломается.